UNIVERSITY of WEST FLORIDA

Machine Learning with Scikit-learn

Regression

Brian Jalaian, Ph.D.
Associate Professor
Intelligent Systems & Robotics Department

Table of Contents

UNIVERSITY of WEST FLORIDA

Linear Regression

Single Linear Regression

UNIVERSITY of WEST FLORIDA

Overview

- **Goal**: To predict a numerical value
- Also called *Univariate Linear Regression*

Terminology

- *table = training set*
  - data used to train the model
- *x = input = feature = output = target = label*
- *(x, y) = training example = data point = instance = sample*
- *m = number of training example*
- $(x^{(i)}, y^{(i)})$ = *i^{th} row of training example*
- *f(x) = function = model = regression line*
- $f_{w,b}(x) = wx + b = \hat{y}$ = *estimated output = predicted output*
- *w = weight*
- *b = bias = intercept = offset*
- $|\hat{y} - y|$ = *offsets = residuals*

	x	y
(1)	223	50
(2)	45	7
(3)	433	786
…	…	…
(47)	92	36

m = 47 $x^{(1)} = 223$ $y^{(1)} = 50$

$(x^{(1)}, y^{(1)}) = (223, 50)$

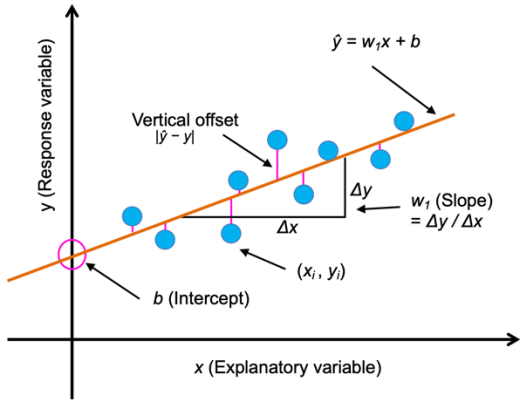


Figure 9.1: A simple one-feature linear regression example

Multiple Linear Regression

UNIVERSITY of WEST FLORIDA

Terminology

- $x_j = j^{th}$ feature
- n = number of features
- $\overrightarrow{x^{(i)}}$ = features of i^{th} training example
- $x_j^{(i)}$ = value of feature j in i^{th} training example

x1	x2	x3	x4	y
1560	3	8	72	100
2501	7	1	75	142
1980	8	4	76	135
1763	1	5	73	146
…	…	…	…	…

- $\overrightarrow{x^{(2)}} = [2501, 7, \mathbf{1}, 75]$
- $x_3^{(2)} = \mathbf{1}$

Number form of features

$$\hat{y} = w_1 x_1 + \cdots + w_n x_n + b$$

Vector form of features

$$\hat{y} = w^\top x + b$$

Matrix form of features

$$\hat{y} = \mathbf{X}w + b$$

- m = *number of training example* (row in table; i^{th} training example)
- n = *number of features* (column in table; j^{th} feature)

- $\mathbf{w} \in \mathbb{R}^n, \mathbf{x} \in \mathbb{R}^n$ (vector includes all features)
- $\hat{y}$ is not a vector

- $\mathbf{X} \in \mathbb{R}^{m \times n}$ (matrix includes training examples & all features)
- $\hat{\mathbf{y}} \in \mathbb{R}^n$ (vector includes all features)

UNIVERSITY of WEST FLORIDA

Cost Function & Gradient Descent

Cost Function for Linear Regression

Cost functions quantify the distance between the real and predicted values of the target

Most common cost function for linear regression: *Mean Squared Error (MSE)*

Goal: Find (w, b): $\widehat{y^{(i)}}$ is close to $y^{(i)}$ for all $\left(x^{(i)}, y^{(i)}\right)$

- $Loss\ function = L\left(\widehat{y^{(i)}}, y^{(i)}\right) = L\left(f_{\vec{w},b}\left(\overrightarrow{x^{(i)}}\right), y^{(i)}\right) = \frac{1}{2}\left(\widehat{y^{(i)}} - y^{(i)}\right)^2 = \frac{1}{2}\left(f_{\vec{w},b}\left(\overrightarrow{x^{(i)}}\right) - y^{(i)}\right)^2$
- $Cost\ function = J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}\left(\widehat{y^{(i)}} - y^{(i)}\right)^2$
  - sum of loss function = cost function
  - recall m = *number of training examples*
  - ½ is just used for convenience to derive the update rule of gradient descent (later)

Revisit our goal: seeking optimal parameters *(w*, b*)* that minimize the total cost across all training examples: $w^*, b^* = \underset{\boldsymbol{w},b}{arg\min} J(\boldsymbol{w}, b)$

→ **How to choose *w* to get minimum cost?**

- Gradient descent

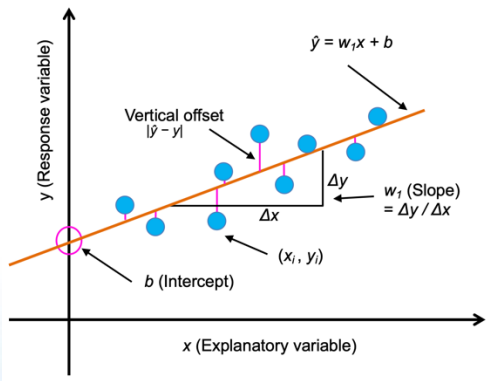


Figure 9.1: A simple one-feature linear regression example

Gradient Descent Algorithm

Outline

- Start with some *(w, b)*
- Keep changing *(w, b)* to reduce *J(w, b)*
- Until settle or near a minimum

How to choose the baby step (direction) ?

- Look around + take the steepest direction

Algorithm

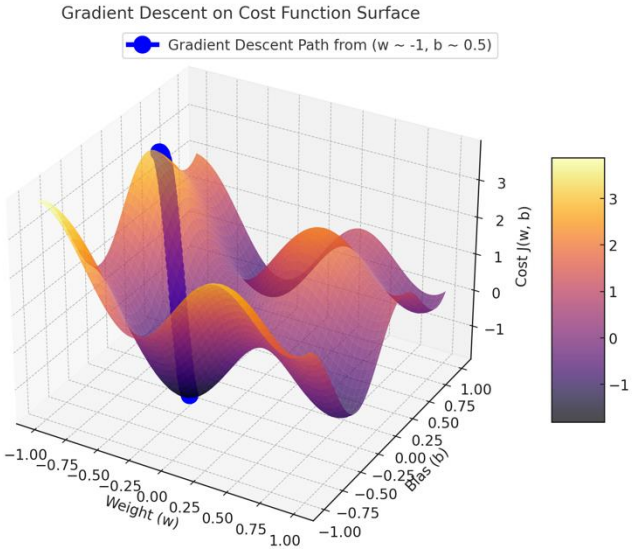
Repeat until convergence (update w and b simultaneously) {

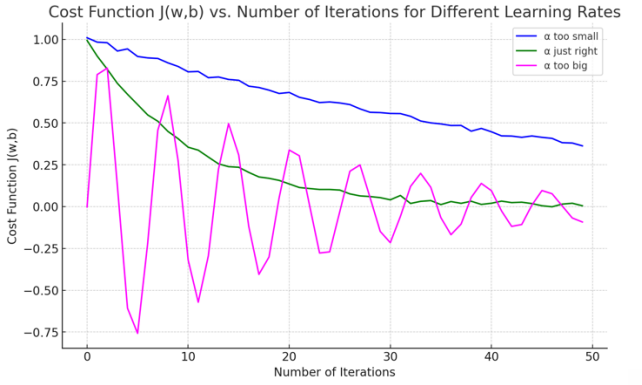
$$w = w - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(f_{w,b}\left(x^{(i)}\right) - y^{(i)} \right) x^{(i)}$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(f_{w,b}\left(x^{(i)}\right) - y^{(i)} \right)$$

}

- α = *learning rate**
  - determines how big the step it's going to take
- $\frac{\partial}{\partial w} J(w,b)$ & $\frac{\partial}{\partial b} J(w,b)$
  - determines what direction it's going to take
- Recall that $f_{w,b}\left(x^{(i)}\right) = wx^{(i)} + b$





learning curve: how α impacts the cost (after iterations, cost should decrease)

*textbook uses η as learning rate

UNIVERSITY of WEST FLORIDA

Robust Fitting Approach - RANSAC

Robust Fitting Approach - RANSAC

Overview

- RANdom SAmple Consensus (RANSAC)
- It fits a regression model to a subset of the data (inliners)
  - since linear regression models can be heavily impacted by the presence of outliers

RANSAC Algorithm

1. Select a random number of examples to be inliers and fit the model.

2. Test all other data points against the fitted model and add those points that fall within a user-given tolerance to the inliers.

3. Refit the model using all inliers.

4. Estimate the error of the fitted model versus the inliers.

5. Terminate the algorithm if the performance meets a certain user-defined threshold or if a fixed number of iterations was reached; go back to step 1 otherwise.

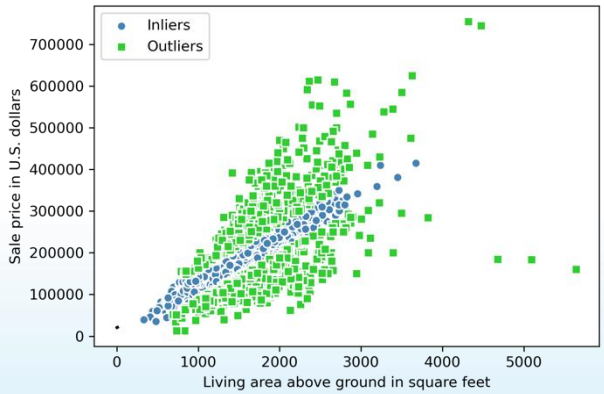


Figure 9.9: Inliers and outliers identified via a RANSAC linear regression model

UNIVERSITY of WEST FLORIDA

Evaluate Linear Regression Models

Overview of Evaluating Models

When evaluating our model, we want to split our dataset into the following datasets:

- **training sets:** fit the model
- **test sets:** evaluate model performance on unseen data to estimate the generalization performance

Problem

When training multiple features regression model, we can't visualize the linear regression hyperplane in a two-dimensional plot.

Solution

Plot the *residuals* (the differences or vertical distances between the actual and predicted values) versus the *predicted values* to diagnose our regression model.

Common ways

- Residual plots
- Mean Squared Error (MSE) & Mean Absolute Error (MAE)
- Coefficient of Determination (R^2)

Residual Plots

Overview

Help to detect nonlinearity and outliers and check whether the errors are randomly distributed.

Example

- **Good regression model:** the errors to be randomly distributed and the residuals to be randomly scattered around the centerline
- **Detect outliers:** the points with a large deviation from the centerline

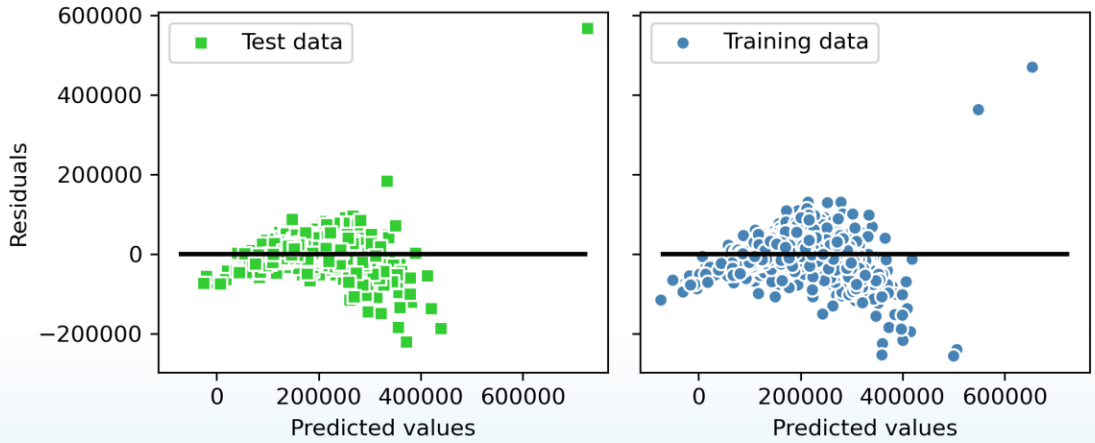


Figure 9.11: Residual plots of our data

MSE & MAE

UNIVERSITY of WEST FLORIDA

Mean Squared Error (MSE)

MSE normalizes according to the sample size, n, in the following formula. This makes it possible to compare across different sample sizes (for example, in the context of learning curves) as well.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}\left(y^{(i)} - \widehat{y^{(i)}}\right)^2$$

Properties

If training set error < test set error, which is an indicator that our model is slightly "overfitting" the training data

Mean Absolute Error (MAE)

More intuitive to show the error on the “original unit scale” than MSE.

$$MAE = \frac{1}{n}\sum_{i=1}^{n}\left|y^{(i)} - \widehat{y^{(i)}}\right|$$

Coefficient of Determination (R^2)

UNIVERSITY of WEST FLORIDA

Overview

It can be understood as a standardized version of the MSE.

R^2 is the fraction of response variance that is captured by the model.

Definition

$$R^2 = 1 - \frac{SSE}{SST}$$

- *SSE = sum of squared errors*:

$$SSE = \sum_{i=1}^{n} \left(y^{(i)} - \widehat{y^{(i)}}\right)^2$$

- *SST = total sum of squares (= variance of response)*:

$$SST = \sum_{i=1}^{n} \left(y^{(i)} - \mu_y\right)^2$$

Properties

- R^2 is bounded between 0 and 1
- $R^2 < 0$, means extreme overfitting, or we forget to scale the test set in the same manner we scaled the training set
- $R^2 = 1$, means the model fits the data perfectly with a corresponding MSE = 0

UNIVERSITY of WEST FLORIDA

Linear Regression Regularization

Linear Regression Regularization

UNIVERSITY of WEST FLORIDA

Overview

Regularization is one approach to tackling the problem of overfitting by adding additional information and thereby shrinking the parameter values of the model to induce a penalty against complexity.

The 3 popular regularization methods for linear regression

- Ridge Regression
- Least Absolute Shrinkage and Selection Operator (LASSO)
- Elastic Net

Ridge Regression

Ridge regression is an L2 penalized model where we simply add the squared sum of the weights to the MSE loss function:

$$L(w)_{\text{Ridge}} = \sum_{i=1}^{n} \left(y^{(i)} - \widehat{y^{(i)}}\right)^2 + \lambda |w|_2^2$$

L2 term is defined as follows:

$$\lambda |w|_2^2 = \lambda \sum_{j=1}^{m} w_j^2$$

Increase $\lambda \rightarrow$ increase regularization strength $\rightarrow$ decrease weights of model

LASSO

Depending on the regularization strength, certain weights can become zero, which also makes LASSO useful as a supervised feature selection technique:

$$L(w)_{\text{Lasso}} = \sum_{i=1}^{n} \left(y^{(i)} - \widehat{y^{(i)}}\right)^2 + \lambda|w|_1$$

L1 penalty for LASSO is defined as the sum of the absolute magnitudes of the model weights, as follows:

$$\lambda||w|_1 = \lambda \sum_{j=1}^{m} \left|w_j\right| |$$

Limitation

It selects at most n features if m > n, where n is the number of training examples.

→ undesirable in certain application of feature selection

→ but avoids saturated models* → generalize well

*saturated models: fit training data perfectly

Elastic Net

Elastic Net is a compromise between ridge regression and LASSO.

It has an L1 penalty to generate sparsity and an L2 penalty such that it can be used for selecting more than n features if m > n:

$$L(w)_{\text{Elastic Net}} = \sum_{i=1}^{n} \left(y^{(i)} - \widehat{y^{(i)}}\right)^2 + \lambda_2 |w|_2^2 + \lambda_1 |w|_1$$

UNIVERSITY of WEST FLORIDA

Polynomial Regression

Polynomial Regression

To model a nonlinear relationship, but it is still considered a multiple linear regression model because of the linear regression coefficients, w.

$$y = w_1 x + w_2 x^2 + \cdots + w_d x^d + b$$

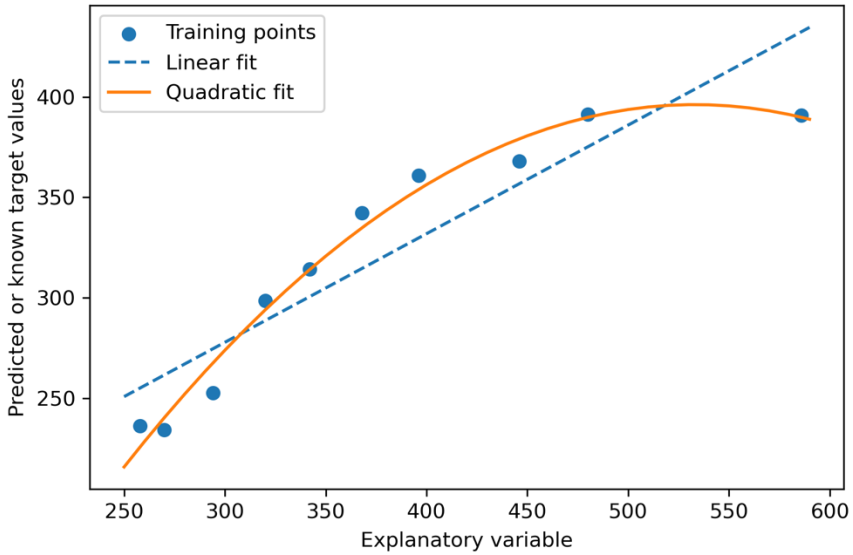


Figure 9.12: A comparison of a linear and quadratic model

UNIVERSITY of WEST FLORIDA

Decision Tree & Random Forest for Regression

Decision Tree for Regression

Overview

- Subdivide the input space into smaller regions that become more manageable.
- Analyze one feature at a time, not taking weighted combinations into account.

Advantage

It works with arbitrary features and does not require any transformation of the features if we are dealing with nonlinear data. (since analyzing one feature at a time)

Goal

Find the feature split that maximizes the *information gain*

= find the feature split that reduces the *impurities* in the child nodes most

Method

We grow a decision tree by iteratively splitting its nodes until the leaves are pure or a stopping criterion is satisfied.

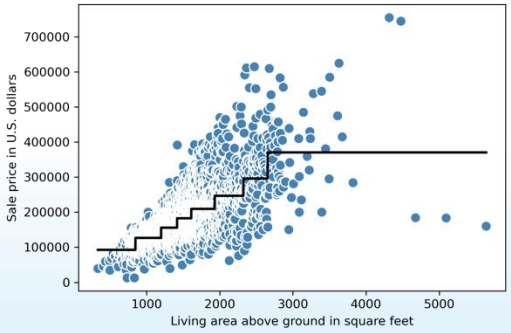


Figure 9.15: A decision tree regression plot

Decision Tree for Regression

UNIVERSITY of WEST FLORIDA

Information gain

Recall the goal: find the feature split that maximizes the information gain.

$$IG(D_p, x_i) = I(D_p) - \frac{N_{\text{left}}}{N_p} I(D_{\text{left}}) - \frac{N_{\text{right}}}{N_p} I\left(D_{\text{right}}\right)$$

- x_i : feature to perform the split
- N_p : number of training examples in the parent node
- I : impurity function
- D_p : subset of training examples at the parent node
- D_{left} and D_{right} : subsets of training examples at the left and right child nodes after the split

Impurity for linear regression

$$I(t) = MSE(t) = \frac{1}{N_t} \sum_{i \in D_t} \left(y^{(i)} - \hat{y}_t\right)^2$$

- N_t : number of training examples at node t
- D_t : training subset at node t
- $y^{(i)}$: true target value
- $\hat{y}_t = \frac{1}{N_t} \sum_{i \in D_t} y^{(i)}$: predicted target value (sample mean)

Random Forest for Regression

UNIVERSITY of WEST FLORIDA

Overview

It is an ensemble technique that combines multiple decision trees.

Advantage

- Usually has a better generalization performance than an individual decision tree due to randomness, which helps to decrease the model's variance.
- They are less sensitive to outliers in the dataset and don't require much parameter tuning. The only parameter in random forests that we typically need to experiment with is **the number of trees** in the ensemble.

Algorithm

1. Draw a random bootstrap sample of size *n*.
(randomly choose *n* examples from the training dataset with replacement)
2. Grow a decision tree with MSE criterion. At each node:
    a. Randomly select *d* features without replacement.
    b. Split the node using the feature that provides the best split according to the objective function, for instance, maximizing the information gain.
3. Repeat steps 1- 2 *k* times.
4. Aggregate the prediction by each tree to calculate the average prediction.

*bootstrap sampling: approximate the distribution of a statistic by resampling with replacement from the original sample